

Software Testing and Validation Lab Assignment

LAB CODE

8CS4-22

Submitted By:

Pranjal Khandelwal

22EARCS125

Under the supervision of

Dr. Krishan Kant Lavania

Professor



Session 2025-26

Department of Computer Science and Engineering
ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, RAJASTHAN

BLACK BOX TESTING

A Comprehensive Study of Specification-Based Software Testing

1. Introduction

so black box testing. basically the whole point is that the tester doesn't look at the code at all. they just test the software from outside.

the reason this method exists is because not everyone who tests software actually knows how to code. in most companies there is a separate QA team and those people are not developers. they don't have access to the codebase and honestly they don't need it either. their job is just to check — does this thing work or not. The process is straightforward. you decide what inputs to give. you run the test. you see what output came. if it matches what was expected then fine, test passed. if something weird came out then that means there is a defect somewhere. simple as that.

I think the best way to understand this is through daily life examples. like when we use any app — Zomato, Paytm, whatever — we are basically doing black box testing without even realising it. we press a button, we expect something to happen. we don't care about the code running behind it. if it works, great. if it doesn't, we complain.

same logic applies here just in a more formal and structured way. the tester is basically acting like a user but in a more systematic manner. they have proper test cases, expected results written down, everything documented.

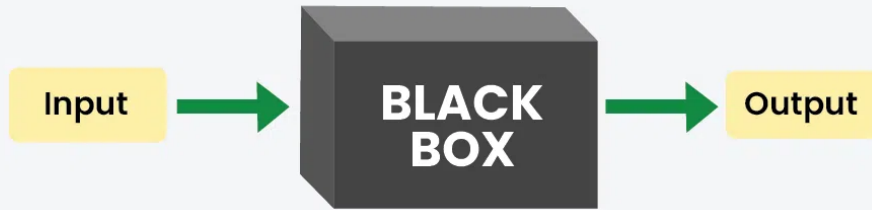
one more thing — this method is really good for checking functional requirements. meaning you can verify whether the software does what the client actually asked for. not how it does it. just whether it does it. and at the end of the day that is what matters most to the client anyway.

2. Core Concept and Definition

Black box testing — some also call it behavioural testing or specification-based testing — is basically a QA technique where you design your test cases purely from the requirements and specs of the software. No source code involved at all. The tester only interacts with the system from outside, through things like the UI, APIs, or command-line inputs. That's the boundary. You don't cross it.

Now the interesting part is how this is different from white box testing. In white box, the tester actually looks at the code — the internal logic, the paths, all of it. Black box is the complete opposite. You have no idea what's written inside and honestly you don't need to. Your only reference is the requirement document or the spec sheet.

Black Box Testing



And because of that, this method doesn't care what language the software is built in. Java, Python, C++, doesn't matter. It also doesn't care about the platform. A web app, a mobile app, an embedded system — black box testing works the same way across all of them. That's what makes it so widely used in real projects, because teams don't have to adjust their testing approach every time the tech stack changes.

3. Black Box Testing Process

The testing process follows a structured sequence of activities, each building upon the previous:

Phase	Activity	Description
Step 1	Requirement Analysis	Study specs & user stories
Step 2	Test Case Design	Define inputs & expected outputs
Step 3	Test Execution	Run tests without code access
Step 4	Result Comparison	Actual vs expected output
Step 5	Defect Reporting	Log bugs & retest after fix

Diagram 1 — Black Box Testing Process Flow

So the process starts with going through the requirement documents, user stories, system specs basically whatever documentation is available. The tester reads all of it carefully because that's the only reference they have. No code to fall back on. After that, test cases are written. Each test case has a specific input and a expected output. Like if the requirement says "user should not be able to login with wrong password" you write a test for that.

Then comes the actual execution part. You run the tests, see what the system does, and compare the results with what was supposed to happen. If something doesn't match — that's a defect. You log it, report it, and hand it over to the development team. Once the developers fix the issue,

testing doesn't just stop there. You run the tests again — this is called regression testing — just to make sure the fix did not accidentally break something else that was working fine before. That happens more often than people think actually.

4. Key Black Box Testing Techniques

Several proven techniques guide test case design in Black Box Testing. Each serves a distinct purpose depending on the nature of the software under examination:

Technique	Purpose	Example
Equivalence Partitioning	Divide input into valid/invalid classes	<i>Age: 0–17, 18–60, 61+</i>
Boundary Value Analysis	Test at input boundaries	<i>Max password length: 7, 8, 9 chars</i>
Decision Table Testing	Test rule combinations	<i>Login: valid/invalid user × valid/invalid pwd</i>
State Transition Testing	Test system state changes	<i>ATM: idle → PIN entry → transaction</i>
Error Guessing	Use experience to predict errors	<i>Empty fields, null inputs, overflow values</i>

Diagram 2 — Black Box Testing Techniques Comparison

Equivalence partitioning is pretty simple in concept. Instead of testing every possible input — which is impossible in most cases — you group similar inputs together. One value from each group is enough. If it works, the rest of the group is assumed to behave the same. Less test cases, same coverage. Boundary value analysis goes one step further. Most bugs don't show up in the middle of a range, they show up at the edges. So, if a field accepts 18 to 60, you test 17, 18, 19 and 59, 60, 61. That's it. The boundaries are where things break.

Decision table testing is for situations where multiple conditions affect the outcome. You list every possible combination of conditions and what the expected result should be. Nothing gets missed, everything is covered systematically. State transition testing is used when the system has different states it moves through. Like a login flow — idle, password entry, authenticated, locked. Each transition needs to be checked. One wrong state change and the whole flow breaks.

Error guessing is the least formal of all five. No structure, no rules. The tester just uses their experience to predict where bugs might be. Empty inputs, null values, weird characters — things that have caused problems before and probably will again.

5. Advantages and Disadvantages

✓ Advantages	✗ Disadvantages
Tester needs no programming knowledge	Limited test coverage — internal paths untested
Unbiased testing from user perspective	Difficult to design tests without source code
Effective for validating functional requirements	Redundant test cases may be written unknowingly
Applicable at all levels of testing	Cannot optimize tests for code structure
Encourages specification-driven thinking	Root cause of bugs harder to pinpoint

Diagram 3 — Advantages vs. Disadvantages of Black Box Testing

The biggest advantage of black box testing is that you don't need to know the code. At all. This means non-technical people can be part of the testing process too — not just developers. QA teams, clients, even end users can test the software without ever opening the codebase. That widens the team significantly.

Also because the testing is done from the outside, it naturally reflects how real users would interact with the system. So the bugs that get caught are usually the ones that actually matter — the ones a user would run into, not some edge case deep inside the code.

But its not perfect. The main problem is coverage. Since you cant see inside the system, you have no way of knowing if every internal path has been tested or not. Some logic buried deep in the code might have a fault that never gets triggered by your test cases. You just wont know.

And even when a bug is found, figuring out exactly where it came from is harder. You know something went wrong, but without code access, narrowing it down takes more time and back and forth with the development team.

6. Real-World Applications

Black box testing shows up in almost every domain. Web development, banking, mobile apps — it's everywhere. In web development its used for things like form submissions, login flows, API responses. The stuff users touch directly. Input goes in, correct response should come out.

Banking is a big use case. Transactions, balance calculations, security checks — all tested from the user side. The tester doesn't care about the backend, they just want the numbers to be right and the system to not break.

Mobile apps use it mainly for UI testing across devices. Same interaction should give same result whether its on a budget Android or a flagship iPhone. When it doesn't, that's a bug.

Where it really stands out though is UAT. User acceptance testing is basically when the client tests the software themselves before signing off. Non-technical people, no code knowledge, just — does this do what we asked for. Black box is the only approach that makes sense here.

Healthcare and finance rely on it heavily too. Regulators and auditors don't have access to source code when they inspect a system. They interact with it like any user would. So if your testing already mirrors that, compliance becomes a lot easier to demonstrate.

7. Conclusion

Black box testing is not going anywhere. It's one of those methods that has proven itself across too many projects and industries to be replaced or ignored. The reason is simple — it keeps the focus on what actually matters to the user. Does the software do what it was supposed to do. That's the question, and black box testing answers it directly. It works at every level too. Unit testing, integration, system testing, UAT — the same approach applies throughout. And with techniques like equivalence partitioning and boundary value analysis already built into the methodology, teams have solid tools to work with without needing to reinvent anything.

The one real weakness is coverage. You can't guarantee that every internal path gets tested. But honestly that problem goes away when you combine black box with white box or grey box testing. Each method covers what the other misses. Together they give a much more complete picture than any single approach could on its own. So, a testing strategy that puts black box at the centre, and fills the gaps with other methods where needed, is probably the most practical and reliable approach for most software projects. It keeps things user focused, accessible to non-technical testers, and grounded in actual requirements — which at the end of the day is what testing is supposed to be about.